

Agenti logici: la logica del prim'ordine

Sintassi, semantica, inferenza

Claudio Gallicchio

a.a. 2023/24

Riferimenti



AIMA Cap 8

- 8.1, 8.2, 8.3

Introduzione alla Logica del Primo Ordine

Verso la Logica del Primo Ordine (First order logic – FOL)

La logica proposizionale (PROP)

- Sufficiente per illustrare concetti base della logica, inferenza e agenti basati su conoscenza
- Limitata nelle capacità di rappresentazione
- Non ha la potenza espressiva per descrivere un ambiente con molti oggetti in modo conciso

La logica del prim'ordine per R.C. (o: Calcolo dei predicati del primo ordine)

- Nella logica dei predicati abbiamo assunzioni ontologiche più ricche
 - ipotesi circa la natura della *realtà* (i.e., *che cosa esiste nel mondo*)
- *gli oggetti, le relazioni, e le proprietà*

La logica del prim'ordine per R.C. (o: Calcolo dei predicati del primo ordine)

Si inizia con una *concettualizzazione*: si tratta di decidere quali sono le cose di cui si vuole parlare

- *Gli oggetti*: un libro, un evento, una persona, un istante di tempo, un insieme, una funzione, un unicorno ...
- Gli oggetti possono essere identificati con simboli o relativamente ad altri oggetti, mediante *funzioni*: *“la madre di Pietro”*
- L'insieme degli oggetti rilevanti costituiscono il *dominio del discorso*. Il dominio potrebbe essere infinito.

La logica del prim'ordine per R.C. (o: Calcolo dei predicati del primo ordine)

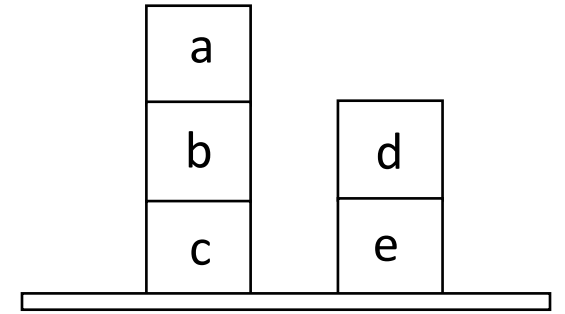
- Le relazioni tra gli oggetti: *“Pietro è amico di Paolo”*
- Le proprietà: *“la madre di Pietro è simpatica”*

N.b.

Proprietà: relazioni unarie

Funzioni: relazioni in cui c'è un solo “valore” per ogni “input”

Esempio: il mondo dei blocchi



Ci interessano i blocchi e alcune loro relazioni spaziali

Dominio: {a, b, c, d, e} $\leftarrow$ *blocchi*

Le funzioni: si individuano le funzioni rilevanti che servono anch'esse per identificare oggetti.

Es. *Hat* la funzione unaria che dato un blocco identifica il blocco che ci sta sopra; $Hat(b)=a$

Le relazioni: si individuano le relazioni interessanti.

Es *On* = {<a, b>, <b, c>, <d, e>}

Clear = {a, d}

Table = {c, e}

Block = {a, b, c, d, e}



Concettualizzazione

$\langle \{a, b, c, d, e\}, \{Hat\}, \{On, Clear, Table, Block\} \rangle$

- Le concettualizzazioni possibili sono infinite: un aspetto importante è il livello di astrazione *giusto* per gli scopi della rappresentazione.

Es. se fosse rilevante il colore o la grandezza dei blocchi dovremmo introdurre predicati anche per questi aspetti

Sintassi della FOL

Simboli e interpretazioni

Gli elementi sintattici di base sono i simboli usati per indicare gli oggetti, le relazioni e le funzioni.

- 3 tipi di simboli:
 - simboli di costante: rappresentano gli oggetti
 - simboli di predicato: rappresentano le relazioni
 - simboli di funzione: rappresentano le funzioni
- Ogni simbolo di predicato e di funzione ha una specifica arità, che ne indica il numero di argomenti

Variabili

- Una variabile è un simbolo che rappresenta elementi arbitrari del dominio
- Sono usate per riferirsi a oggetti non specificati
 - consentono la formulazione di predicati che possono essere veri o falsi a seconda dell'assegnazione di valori specifici alle variabili
- Sono usate in predicati, funzioni, e quantificatori
- Solitamente indicate con lettere minuscole (x , y , ...)

Il linguaggio: i termini

Un **termine** è un'espressione logica che **si riferisce a un oggetto**

- La sintassi dei termini:

$Termine \rightarrow Costante \mid Variabile \mid Funzione(Termine, \dots)$

(un numero di termini pari alla arità della funzione)

- Esempi di termini ben formati:

$f(x, y)$	$+(2, 3)$
$Padre-di(\text{Giovanni})$	$x, A, B, 2$
$Prezzo(\text{Banane})$	$\hat{H}at(A)$

Uguaglianza

- Il simbolo di uguaglianza $=$ si usa per affermare che due termini fanno riferimento allo stesso oggetto.
 - es. $Padre(Claudio) = Giampiero$
- Si può usare per esprimere fatti riguardanti una data funzione
 - es. $\exists x, y \text{ Fratello}(x, Riccardo) \wedge \text{Fratello}(y, Riccardo) \wedge \neg(x = y)$

Il linguaggio: le formule atomiche

- Una formula atomica è l'espressione più semplice e indivisibile che afferma una relazione tra oggetti del dominio di discorso.
- È composta da un predicato seguito da una lista di termini (variabili, costanti, o la combinazione di entrambi) che corrispondono all'arità del predicato.
- Le formule atomiche non contengono quantificatori, connettivi logici, o altre formule come componenti.

Il linguaggio: le formule atomiche

Una **formula atomica** (o atomo) è **composta da un simbolo di predicato seguito da una lista di termini tra parentesi.**

La sintassi delle formule atomiche:

Formula-atomica $\rightarrow$ *True* | *False* |

Termine = *Termine* |

Predicato (*Termine*, ...)

(un numero di termini pari alla arità del predicato)

Il linguaggio: le formule complesse

Possiamo usare i connettivi logici (e i quantificatori) per costruire formule complesse (o composte)

Formula $\rightarrow$ *Formula-atomica* |
 Formula *Connettivo* *Formula* |
 Quantificatore Variabile Formula |
 $\neg$ *Formula* | (*Formula*)

Il linguaggio: formule ben formate

Esempi di **formule atomiche**:

Ama(Giorgio, Lucia) $+(2, 3) = 5$

On(A, B) $x = 5$

Madre-di(Luigi) = Silvana

Amico(*Padre-di*(Giorgio), *Padre-di*(Elena))

Esempi di **formule complesse**:

$On(A, B) \wedge On(B, C)$ (*coniunzione*)

$Studia(Paolo) \Rightarrow Promosso(Paolo)$ (*implicazione*)

Il linguaggio: quantificatori

- Quantificatore universale

- La relazione si applica a tutti gli elementi del dominio
- $\forall x \text{ Ama}(x, \text{Gelato})$

- Quantificatore esistenziale

- la relazione si applica ad almeno un elemento del dominio
- $\exists x \text{ Mela}(x) \wedge \text{Rossa}(x)$

Il linguaggio: quantificatori

- Nota: l'ordine dei quantificatori è importante:

- $\forall x (\exists y \text{ Ama}(x, y))$

Tutti amano qualcuno

- $\exists y (\forall x \text{ Ama}(x, y))$

Esiste qualcuno amato da tutti

- Ambito (scope) dei quantificatori:

ambito di $\exists y$

$$\forall x (\exists y \text{ Ama}(x, y))$$

ambito di $\forall x$

$$\forall x (\exists y \text{ Ama}(x, y))$$

La logica dei predicati del prim'ordine (FOL)

■ Il linguaggio: vocabolario

- *Connettivo* $\rightarrow \wedge \mid \vee \mid \neg \mid \Rightarrow \mid \Leftrightarrow \mid \Leftarrow$
- *Quantificatore* $\rightarrow \forall \mid \exists$
- *Variabile* $\rightarrow x \mid y \mid \dots a \mid s \dots$ (lettere minuscole)
- *Costante* $\rightarrow$ Es. $A \mid B \mid \dots$ Mario $\mid$ Pippo $\mid$ 2 $\dots$ (lettere maiuscole)
- *Funzione* $\rightarrow$ Es. $Hat \mid Padre-di \mid + \mid - \mid \dots$
(con *arità* ≥ 1) 1 1 2 2
- *Predicato* $\rightarrow$ Es. $On \mid Clear \mid \geq \mid < \dots$
(con *arità* ≥ 0) 2 1 2 2

Variabili legate e Variabili libere

usate con un quantificatore

non usate con un quantificatore

- Di solito le variabili sono usate nell'ambito di quantificatori. In tal caso le *occorrenze* si dicono *legate*.
- Se non legate si dicono *libere*.

$Mela(x) \Rightarrow Rossa(x)$

x è libera in entrambe le occorrenze

$\forall x Mela(x) \Rightarrow Rossa(x)$

x è legata ...

$Mela(x) \Rightarrow \exists y Rossa(y)$

la prima è libera, la seconda legata

Formule *chiuse*, *aperte*, *ground*

- *Def. Formula **chiusa***: una formula che **non contiene** occorrenze di **variabili libere**.
- *Def. Formula **aperta***: una formula che **contiene** occorrenze di **variabili libere**.
- *Def. Formula **ground***: una formula che **non contiene** **variabili**.

Il linguaggio: precedenza tra gli operatori

Precedenza tra gli operatori logici:

$= > \neg > \wedge > \vee > \Rightarrow, \Leftrightarrow > \forall, \exists$

Es. $\forall x \text{ Persona}(x) \Rightarrow \text{Sesso}(x)=M \vee \text{Sesso}(x)=F$

è da interpretare come ...

$\forall x \text{ Persona}(x) \Rightarrow (\text{Sesso}(x)=M) \vee (\text{Sesso}(x)=F)$ $=$

$\forall x \text{ Persona}(x) \Rightarrow ((\text{Sesso}(x)=M) \vee (\text{Sesso}(x)=F))$ $\vee$

$\forall x (\text{Persona}(x) \Rightarrow ((\text{Sesso}(x)=M) \vee (\text{Sesso}(x)=F)))$ $\forall$

Il linguaggio: precedenza tra gli operatori

Precedenza tra gli operatori logici:

$$= > \neg > \wedge > \vee > \Rightarrow, \Leftrightarrow > \forall, \exists$$

Es. $\forall x \text{ Persona}(x) \Rightarrow \text{Sesso}(x)=M \vee \text{Sesso}(x)=F \vee \text{Sesso}(x)=NB \vee \text{Sesso}(x)=GQ \vee \text{Sesso}(x)=GF \vee \text{Sesso}(x)=A$

è da interpretare come ...

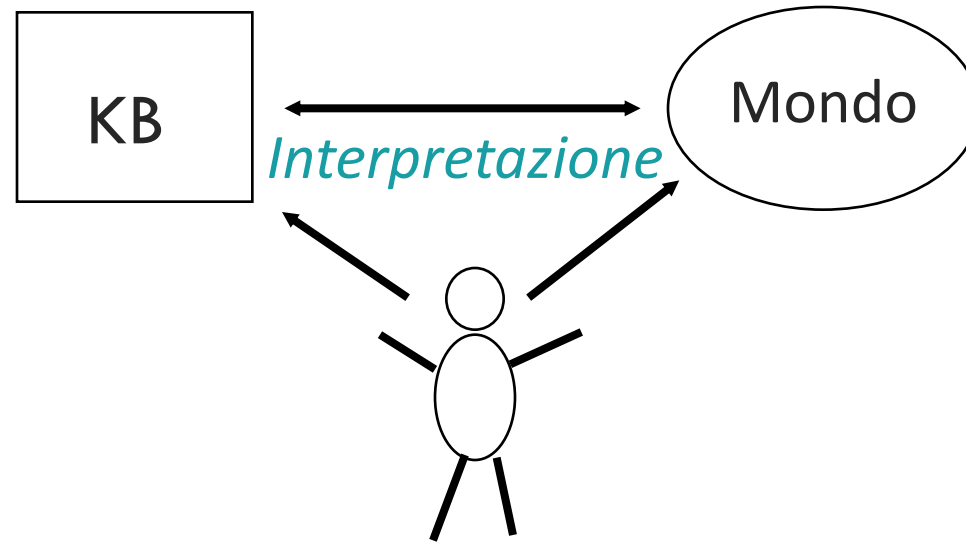
$\forall x \text{ Persona}(x) \Rightarrow (\text{Sesso}(x)=M) \vee (\text{Sesso}(x)=F) \vee (\text{Sesso}(x)=NB) \vee (\text{Sesso}(x)=GQ) \vee (\text{Sesso}(x)=GF) \vee (\text{Sesso}(x)=A)$

$\forall x \text{ Persona}(x) \Rightarrow ((\text{Sesso}(x)=M) \vee (\text{Sesso}(x)=F) \vee (\text{Sesso}(x)=NB) \vee (\text{Sesso}(x)=GQ) \vee (\text{Sesso}(x)=GF) \vee (\text{Sesso}(x)=A))$

$\forall x (\text{Persona}(x) \Rightarrow ((\text{Sesso}(x)=M) \vee (\text{Sesso}(x)=F) \vee (\text{Sesso}(x)=NB) \vee (\text{Sesso}(x)=GQ) \vee (\text{Sesso}(x)=GF) \vee (\text{Sesso}(x)=A)))$

Semantica della FOL

Semantica dichiarativa



Consiste nello stabilire una corrispondenza tra:

- i termini del linguaggio e gli oggetti del mondo
- le formule chiuse e i valori di verità

Componenti principali

■ Universo del discorso

- un insieme non vuoto di elementi, che rappresenta l'insieme di tutti gli oggetti che si prendono in considerazione.

■ Assegnazione di Valori

- Alle **costanti** si assegna un elemento specifico dell'universo di discorso.
- Alle **variabili** si possono assegnare elementi dell'universo di discorso attraverso una funzione di assegnazione.
- Alle **funzioni** si assegna una mappatura da una sequenza di elementi dell'universo a un singolo elemento dell'universo (rispettando l'arietà della funzione).
- Ai **predicati** si assegna una relazione sull'universo di discorso (anch'essa rispettando l'arietà del predicato).

Componenti principali

■ Verità di Formule

- Una **formula atomica** è vera se l'interpretazione assegna ai suoi termini una sequenza che rientra nella relazione specificata dal predicato.
- Le **formule complesse** sono valutate sulla base dei loro componenti usando il significato dei connettivi logici (come $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$) e dei quantificatori ($\forall$, $\exists$).

Interpretazione

Una interpretazione I stabilisce una corrispondenza precisa tra elementi atomici del linguaggio ed elementi della concettualizzazione.

I interpreta:

- i simboli di costante come elementi del dominio D
- i simboli di funzione come funzioni da n -uple di D in D
- i simboli di predicato come insiemi di n -uple (relazioni)

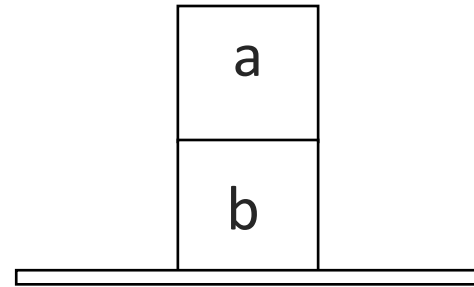
Semantica: un esempio

On(A, B)

Clear (A)

Table (B)

Due interpretazioni possibili:



... quella intesa

$I(A)=a$

$I(B)=b$

$I(On)=\{<a, b>\}$

$I(Clear)=\{a\}$

$I(Table)=\{b\}$

... un'altra possibile

$I(A)=a$

$I(B)=b$

$I(On)=\{<b, a>\}$

$I(Clear)=\{b\}$

$I(Table)=\{a\}$

Semantica composizionale

- Il significato di un termine o di una formula composta è determinato in funzione del significato dei suoi componenti:
 - Es. Sorella(Madre(Pietro), Maria)
 - La formula $A \wedge B$ è vera in una certa interpretazione se entrambe A e B sono vere
 - $\neg A$ è vera se A è falsa
 - $A \vee B$ è vera se A è vera oppure B è vera (o entrambe)
 - $A \Rightarrow B$ è vera se A è falsa oppure B è vera (come $\neg A \vee B$)

Semantica ($\forall$)

- $\forall x A(x)$ è vera se lo è per ciascun elemento del dominio di A

- Se il dominio è finito equivale a un grosso $\wedge$

$\forall x \text{Mortale}(x)$

$\text{Mortale}(\text{Gino}) \wedge \text{Mortale}(\text{Pippo}) \wedge \dots$

- Tipicamente $\forall$ si usa insieme a $\Rightarrow$

$\forall x \text{Persona}(x) \Rightarrow \text{Mortale}(x)$

Difficilmente una proprietà è universale; le condizioni nell'antecedente restringono la portata dell'asserzione e la qualificano.

Semantica ($\exists$)

- $\exists x A(x)$ è vera se esiste almeno un elemento del dominio per cui A è vera
- Se il dominio è finito equivale a un grosso $\vee$
 $\exists x \text{Persona}(x)$
 $\text{Persona}(\text{Gino}) \vee \text{Persona}(\text{Pippo}) \vee \dots$
- Tipicamente $\exists$ si usa con $\wedge$
 $\exists x \text{Persona}(x) \wedge \text{Speciale}(x)$
 $\exists x \text{Persona}(x) \Rightarrow \text{Speciale}(x)$
Tropo debole: non sto nemmeno asserendo che esista una persona.

Relazione tra $\forall$ e $\exists$

Da qui discendono delle proprietà che mettono in relazione $\forall$ e $\exists$.

$$\forall x \neg P(x) \equiv \neg \exists x P(x)$$

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

$$\forall x P(x) \equiv \neg \exists x \neg P(x)$$

$$\neg \forall x \neg P(x) \equiv \exists x P(x)$$

$$\neg P \wedge \neg Q \equiv \neg (P \vee Q)$$

$$\neg (P \wedge Q) \equiv \neg P \vee \neg Q$$

$$P \wedge Q \equiv \neg (\neg P \vee \neg Q)$$

$$P \vee Q \equiv \neg (\neg P \wedge \neg Q)$$

Semantica “standard” e semantica “database”

- *Riccardo ha solo due fratelli: Giovanni e Goffredo.* In logica classica:

$\text{Fratello}(\text{Riccardo}, \text{Giovanni}) \wedge \text{Fratello}(\text{Riccardo}, \text{Goffredo})$

$\wedge \text{Giovanni} \neq \text{Goffredo}$

$\wedge \forall x \text{Fratello}(\text{Riccardo}, x) \Rightarrow (x = \text{Giovanni}) \vee (x = \text{Goffredo})$

- Semantica dei database (e di alcuni linguaggi per la RC)
 - **Ipotesi dei nomi unici**: simboli distinti, oggetti distinti
 - ✓ ogni costante fa riferimento ad un oggetto distinto
 - **Ipotesi del mondo chiuso**: tutto ciò di cui non si sa che è vero è falso
 - ✓ le formule atomiche non conosciute come vere sono false
 - **Chiusura del dominio**: esistono solo gli oggetti di cui si parla
 - ✓ ogni modello contiene un numero di elementi del dominio non superior a quello degli elementi denominati dai simboli di costante

Interazione con la KB in FOL

- Asserzioni

TELL(KB, $Re(Giovanni)$), TELL(KB, $Re(Giorgio)$),

TELL(KB, $\forall x Re(x) \Rightarrow Persona(x)$)

- Interrogazioni (*queries*)

ASK(KB, $Persona(Giovanni)$) Sì, se $KB \models Persona(Giovanni)$

ASK(KB, $\exists x Persona(x)$)

- ✓ 'Sì' sarebbe riduttivo

- ✓ La risposta è una lista di sostituzioni o legami: $[\{x/Giovanni\} \{x/Giorgio\} \dots]$

Esempio: mondo del Wumpus

- Percezioni: possiamo usare un predicato binario $Percezione(5\text{-upla}, t)$

$Percezione([Fetore, Brezza, Scintillio, None, None], 5)$

- Regole generali per le percezioni

$\forall t, s, b, m, c \text{ } Percezione([s, b, Scintillio, m, c], t) \Rightarrow Scintillio(t)$

$\forall t, s, b, m, c \text{ } Percezione([s, b, None, m, c], t) \Rightarrow \neg Scintillio(t)$

- Descrizione della mappa

$$\forall x, y, a, b \text{ } Adiacente([x, y], [a, b]) \Leftrightarrow (x=a \wedge (y=b-1 \vee y=b+1)) \vee$$
$$(y=b \wedge (x=a-1 \vee x=a+1))$$

e molto altro ancora...

Inferenza nella logica del prim'ordine

- Riduzione a inferenza proposizionale
- Il *metodo di risoluzione* per FOL
 - Trasformazione in forma a clausole
 - Unificazione

Riferimenti



AIMA Cap 9

■ 9.1, 9.2

Inferenza proposizionale & Inferenza del primo ordine

1. Convertire la KB del primo ordine in logica proposizionale
2. Usare l'inferenza proposizionale

...Dobbiamo eliminare i quantificatori

Istanziamento Universale

- Possiamo inferire tutte le formule ottenute sostituendo un termine ground a una variabile quantificata universalmente.
 - termine ground: formula che non contiene variabili
- Sostituzione: $[x/g]$ risultato della sostituzione di g per x
 - $A[x/g]$: sostituiamo g per x in A
 - in ALMA: $\text{Subst}(\{x/g\}, A)$

Regole di inferenza per $\forall$

- Istanziamento dell'Universale ($\forall$ -eliminazione)

$$\frac{\forall x A[x]}{A[x/g]}$$

dove g è un termine *ground* e $A[x/g]$ è il risultato della sostituzione di g per x in A .

- Da: $\forall x \text{Re}(x) \wedge \text{Avido}(x) \Rightarrow \text{Malvagio}(x)$ si possono ottenere
 - $\text{Re}(\text{Giovanni}) \wedge \text{Avido}(\text{Giovanni}) \Rightarrow \text{Malvagio}(\text{Giovanni})$
 - $\text{Re}(\text{Padre}(\text{Giovanni})) \wedge \text{Avido}(\text{Padre}(\text{Giovanni})) \Rightarrow \text{Malvagio}(\text{Padre}(\text{Giovanni}))$

Regole per l'esistenziale ($\exists$)

Sostituire una variabile quantificata esistenzialmente con un unico nuovo simbolo costante

Regole per l'esistenziale ($\exists$)

Sostituire una variabile
quantificata esistenzialmente
con un unico nuovo simbolo
costante

- Istanziamento dell'esistenziale ($\exists$ -eliminazione)

$$\exists x A[x]$$

$$A[x/k]$$

1. se $\exists$ non compare nell'ambito di $\forall$, k è una costante nuova (*costante di Skolem*)
2. altrimenti va introdotta una funzione (di Skolem) nelle variabili quantificate universalmente

$$\exists x \text{Padre}(x, G)$$

diventa

$$\text{Padre}(K, G)$$

$$\forall x \exists y \text{Padre}(x, y)$$

diventa

$$\forall x \text{Padre}(x, p(x))$$

e non

$$\forall x \text{Padre}(x, K)$$

... altrimenti tutti avrebbero lo stesso padre !

Riduzione a inferenza proposizionale

- Proposizionalizzazione (*Grounding*)
 - Creare tante istanze delle formule quantificate universalmente quanti sono gli oggetti menzionati
 - Eliminare i quantificatori esistenziali skolemizzando
 - Sostituire le formule atomiche ground con simboli proposizionali
 - ✓ es.: $\text{Re}(\text{Giovanni})$ diventa ReGiovanni

Riduzione a inferenza proposizionale

A questo punto possiamo trattare la KB come proposizionale e applicare gli algoritmi visti. Problemi?

- Le costanti sono in numero finito ...
- ... ma se ci sono funzioni, il numero di istanze da creare è infinito:
Giovanni, Padre(Giovanni), Padre(Padre(Giovanni)) ...

Teorema di Herbrand

Se $KB \models \alpha$ allora c'è una dimostrazione che coinvolge solo un sotto-insieme finito della KB proposizionalizzata

Teorema di Herbrand

Se $KB \models \alpha$ allora c'è una dimostrazione che coinvolge solo un sotto-insieme finito della KB proposizionalizzata

- Si può procedere incrementalmente ...
 1. Creare le istanze con le costanti
 2. Creare poi quelle con un solo livello di annidamento Padre(Giovanni), Madre(Giovanni)
 3. Poi quelle con due livelli di annidamento Padre(Padre(Giovanni)), Padre(Madre(Giovanni)) Madre(Padre(Giovanni)) Madre(Madre(Giovanni)) ...

...finchè non siamo in grado di costruire la dimostrazione proposizionale della formula che è conseguenza logica
- Se $KB \not\models \alpha$ il processo non termina. Il problema è *semidecidibile*.

Metodo di risoluzione per il FOL

- Abbiamo visto la regola di risoluzione per PROP: un metodo deduttivo corretto e completo con un'unica regola
- Possiamo estendere al FOL il metodo di risoluzione?
- Sì. Ma per arrivare a definire la regola ...
 1. Dobbiamo estendere al FOL la trasformazione in forma a clausole
 2. Dobbiamo introdurre il concetto di **unificazione**

Forma a clausole

- Costanti, funzioni, predicati sono come definiti (ma escludiamo nel seguito formule atomiche del tipo $t_1=t_2$)
- Una clausola è un insieme di **letterali**, che rappresenta la loro disgiunzione

Clausola $\rightarrow \{Letterale, \dots, Letterale\}$

Letterale $\rightarrow Formula_atomica \mid \neg Formula_atomica$

- Una KB è un insieme di clausole.

Trasformazione in forma a clausole

Teorema: per ogni formula chiusa α del FOL è possibile trovare in maniera *effettiva* un insieme di clausole $FC(\alpha)$ che è *soddisfacibile* sse α lo è [*insoddisfacibile* sse α lo è]

- effettivo = esiste una procedura che può essere eseguita per trasformare la formula α in un insieme di clausole $FC(\alpha)$ tale che valga questo risultato.
- per ogni formula chiusa del FOL, esiste un metodo sistematico per convertirla in un insieme di clausole tale che la soddisfacibilità della formula originale è preservata.

Trasformazione in forma a clausole - Esempio

Vediamo la trasformazione in dettaglio per la frase

*“Tutti coloro che amano tutti gli animali
sono amati da qualcuno”*

$$\forall x (\forall y \text{ Animale}(y) \Rightarrow \text{Ama}(x,y)) \Rightarrow (\exists y \text{ Ama}(y, x))$$

Trasformazione: passo 1

1. Eliminazione delle implicazioni ($\Rightarrow$ e $\Leftrightarrow$):

$A \Rightarrow B$ diventa $\neg A \vee B$

$A \Leftrightarrow B$ diventa $(\neg A \vee B) \wedge (\neg B \vee A)$

$\forall x (\forall y \text{ Animale}(y) \Rightarrow \text{Ama}(x,y)) \Rightarrow (\exists y \text{ Ama}(y, x))$

$\forall x \neg(\forall y \text{ Animale}(y) \Rightarrow \text{Ama}(x,y)) \vee (\exists y \text{ Ama}(y, x))$

$\forall x \neg(\forall y \neg \text{Animale}(y) \vee \text{Ama}(x,y)) \vee (\exists y \text{ Ama}(y, x))$

Trasformazione: passo 2

2. Negazioni all'interno

$\neg\neg A$ diventa A

$\neg(A \wedge B)$ diventa $\neg A \vee \neg B$ (De Morgan)

$\neg(A \vee B)$ diventa $\neg A \wedge \neg B$ (De Morgan)

$\neg\forall x A$ diventa $\exists x \neg A$

$\neg\exists x A$ diventa $\forall x \neg A$

$\forall x \neg(\forall y \neg \text{Animale}(y) \vee \text{Ama}(x, y)) \vee (\exists y \text{Ama}(y, x))$

$\forall x (\exists y \neg(\neg \text{Animale}(y) \vee \text{Ama}(x, y))) \vee (\exists y \text{Ama}(y, x))$

$\forall x (\exists y (\neg\neg \text{Animale}(y) \wedge \neg \text{Ama}(x, y))) \vee (\exists y \text{Ama}(y, x))$

$\forall x (\exists y (\text{Animale}(y) \wedge \neg \text{Ama}(x, y))) \vee (\exists y \text{Ama}(y, x))$

Trasformazione: passo 3

3. Standardizzazione delle variabili: facciamo in modo che ogni quantificatore usi una variabile diversa

$$\forall x (\exists y (\text{Animale}(y) \wedge \neg \text{Ama}(x, y))) \vee (\exists y \text{ Ama}(y, x))$$

$$\forall x (\exists y (\text{Animale}(y) \wedge \neg \text{Ama}(x, y))) \vee (\exists z \text{ Ama}(z, x))$$

Trasformazione: passo 4

4. Skolemizzazione: eliminazione dei quantificatori esistenziali

$$\forall x (\exists y (\text{Animale}(y) \wedge \neg \text{Ama}(x, y)) \vee (\exists z \text{Ama}(z, x)))$$

Ci sono due quantificatori esistenziali nell'ambito di uno universale, dobbiamo introdurre due funzioni di Skolem

$$\forall x (\text{Animale}(F(x)) \wedge \neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x))$$

Trasformazione: passo 5

5. Eliminazione quantificatori universali

- Possiamo portarli tutti davanti (forma *prenessa*)

$(\forall x A) \vee B$ diventa $\forall x (A \vee B)$

$(\forall x A) \wedge B$ diventa $\forall x (A \wedge B)$

equivalente se B non contiene x

- ... e poi eliminarli usando la *convenzione* che le variabili libere sono quantificate universalmente

$\forall x (\text{Animale}(F(x)) \wedge \neg \text{Ama}(x, F(x))) \vee \text{Ama}(G(x), x)$ diventa

$(\text{Animale}(F(x)) \wedge \neg \text{Ama}(x, F(x))) \vee \text{Ama}(G(x), x)$

Trasformazione: passo 6

6. Forma normale congiuntiva (congiunzione di disgiunzioni di letterali):

$A \vee (B \wedge C)$ diventa $(A \vee B) \wedge (A \vee C)$

$(\text{Animale}(F(x)) \wedge \neg \text{Ama}(x, F(x))) \vee \text{Ama}(G(x), x)$

$(\text{Animale}(F(x)) \vee \text{Ama}(G(x), x)) \wedge (\neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x))$

Trasformazione: passo 7

7. Notazione a clausole:

$(\text{Animale}(F(x)) \vee \text{Ama}(G(x), x)) \wedge (\neg \text{Ama}(x, F(x)) \vee \text{Ama}(G(x), x))$

$\{\text{Animale}(F(x)), \text{Ama}(G(x), x)\}$

$\{\neg \text{Ama}(x, F(x)), \text{Ama}(G(x), x)\}$

Trasformazione: passo 8

8. Separazione delle variabili: clausole diverse, variabili diverse

Nota: $\forall x (P(x) \wedge Q(x)) \Leftrightarrow \forall x_1 P(x_1) \wedge \forall x_2 Q(x_2)$

$\{\text{Animale}(F(x)), \text{Ama}(G(x), x)\} \rightarrow \{\text{Animale}(F(x_1)), \text{Ama}(G(x_1), x_1)\}$

$\{\neg \text{Ama}(x, F(x)), \text{Ama}(G(x), x)\} \rightarrow \{\neg \text{Ama}(x_2, F(x_2)), \text{Ama}(G(x_2), x_2)\}$

NOTA: tutti i passi meno la Skolemizzazione preservano l'equivalenza delle formule. È comunque preservata la *soddisfacibilità*

Una formula skolemizzata è soddisfacibile esattamente quando lo è la formula originale

Unificazione: definizione

- *Unificazione*: operazione mediante la quale si determina se due espressioni possono essere rese **identiche** mediante una *sostituzione* di termini a variabili
- Il risultato è la **sostituzione** che rende le due espressioni identiche, detta *unificatore*, o FAIL, se le espressioni non sono unificabili

Sostituzione

Sostituzione: un insieme finito di associazioni tra variabili e termini, in cui ogni variabile compare una sola volta sulla sinistra.

Es. $\{x_1/A, x_2/f(x_4), x_3/B\}$

Il significato è che A va sostituita a x_1 , $f(x_4)$ va sostituito a x_2 , B a x_3

Es. $\{x/g(y), z/f(y)\}$

Sostituzione

Sostituzione: un insieme finito di associazioni tra variabili e termini, in cui ogni variabile compare una sola volta sulla sinistra.

Es. $\{x_1/A, x_2/f(x_4), x_3/B\}$

Il significato è che A va sostituita a x_1 , $f(x_4)$ va sostituito a x_2 , B a x_3

Es. $\{x/g(y), z/f(y)\}$

Nota: sulla sinistra solo variabili, sulla destra costanti, variabili, funzioni ...
con la restrizione che una variabile sulla sinistra non può comparire anche sulla destra.

- $\{x/f(x)\}$ NO, sostituzione circolare
- $\{x/g(y), y/z\}$ NO, non normalizzata $\{x/g(z), y/z\}$

Applicazione di sostituzione

Sia σ una sostituzione, A un'espressione:

- $A\sigma$ istanza generata dalla sostituzione (delle variabili con le corrispondenti espressioni)

Applicazione di sostituzione

Sia σ una sostituzione, A un'espressione:

- $A\sigma$ istanza generata dalla sostituzione (delle variabili con le corrispondenti espressioni)
- In ALMA SUBST(σ, A)

Esempio

$$\text{SUBST}(\{x/A, y/f(B), z/w\}, P(x, x, y, v)) = P(A, A, f(B), v)$$

$$\text{SUBST}(\{x/g(y), y/z, z/f(x)\}, Q(x, y, z)) = Q(g(y), z, f(x)) \text{ non normalizzata}$$

Nota: le variabili vengono sostituite **simultaneamente** e si esegue un solo passo di sostituzione

Espressioni unificabili

- *Espressioni unificabili*: se esiste una sostituzione che le rende **identiche** (*unificatore*) oppure FAIL

Es. $P(A, y, z)$ e $P(x, B, z)$ sono unificabili con $\tau = \{x/A, y/B, z/C\}$

Espressioni unificabili

- *Espressioni unificabili*: se esiste una sostituzione che le rende **identiche** (*unificatore*) oppure FAIL

Es. $P(A, y, z)$ e $P(x, B, z)$ sono unificabili con $\tau = \{x/A, y/B, z/C\}$

- τ è un unificatore, ma non l'unico ... un altro è ad esempio $\sigma = \{x/A, y/B\}$
- σ è *più generale* di τ (istanza 'meno')
- vorremmo *l'unificatore più generale* di tutti (*Most General Unifier* - MGU)
- *Teorema*: l'unificatore più generale è unico, a parte i nomi delle variabili (l'ordine non conta).

Algoritmo di unificazione

- L'algoritmo di unificazione prende in input due espressioni p, q e restituisce un MGU θ se esiste
 - $\text{UNIFY}(p, q) = \theta$ tale che $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$
 - altrimenti FAIL
- L'algoritmo esplora in parallelo le due espressioni e costruisce l'unificatore strada facendo
- Appena trova espressioni non unificabili fallisce
- Una causa di fallimento sono sostituzioni del tipo $x = f(x)$; questo controllo si chiama *occurr check*

Algoritmo di unificazione (AIMA)

```
function Unify( $x, y, \theta = \text{vuoto}$ ) returns una sostituzione che rende  $x$  e  $y$  identici, o fallimento  
    if  $\theta = \text{fallimento}$  then return fallimento  
    else if  $x = y$  then return  $\theta$                                 // caso di successo  
    else if Variabile?( $x$ ) then return Unify-Var( $x, y, \theta$ )  
    else if Variabile?( $y$ ) then return Unify-Var( $y, x, \theta$ )  
    else if Composta?( $x$ ) and Composta?( $y$ ) then                // es.  $\text{Op}(F(A,B)) = F$    $\text{Args}(F(A,B))=(A,B)$   
        return Unify( $\text{Args}(x), \text{Args}(y), \text{Unify}(\text{Op}(x), \text{Op}(y), \theta)$ )  
    else if Lista?( $x$ ) and Lista?( $y$ ) then  
        return Unify( $\text{Resto}(x), \text{Resto}(y), \text{Unify}(\text{Primo}(x), \text{Primo}(y), \theta)$ )  
    else return fallimento
```

Algoritmo di unificazione (AIMA)

function Unify($x, y, \theta = \text{vuoto}$) **returns** una sostituzione che rende x e y identici, o *fallimento*

if $\theta = \text{fallimento}$ **then return** *fallimento*

else if $x = y$ **then return** θ

else if Variabile?(x) **then return** Unify-Var(x, y, θ)

else if Variabile?(y) **then return** Unify-Var(y, x, θ)

else if Composta?(x) **and** Composta?(y) **then**

return Unify(Args(x), Args(y), Unify(Op(x), Op(y), θ))

else if Lista?(x) **and** Lista?(y) **then**

return Unify(Resto(x), Resto(y), Unify(Primo(x), Primo(y), θ))

else return *fallimento*

Algoritmo di unificazione (AIMA)

```
function Unify( $x, y, \theta = \text{vuoto}$ ) returns una sostituzione che rende  $x$  e  $y$  identici, o fallimento  
  if  $\theta = \text{fallimento}$  then return fallimento  
  else if  $x = y$  then return  $\theta$   
  else if Variabile?( $x$ ) then return Unify-Var( $x, y, \theta$ )  
  else if Variabile?( $y$ ) then return Unify-Var( $y, x, \theta$ )  
  else if Composta?( $x$ ) and Composta?( $y$ ) then  
    return Unify(Args( $x$ ), Args( $y$ ), Unify(Op( $x$ ), Op( $y$ ),  $\theta$ ))  
  else if Lista?( $x$ ) and Lista?( $y$ ) then  
    return Unify(Resto( $x$ ), Resto( $y$ ), Unify(Primo( $x$ ), Primo( $y$ ),  $\theta$ ))  
  else return fallimento
```

x e y possono essere qualsiasi espressione (costante, variabile, espressione composta, ...)

Algoritmo di unificazione (AIMA)

```
function Unify( $x, y, \theta = \text{vuoto}$ ) returns una sostituzione che rende  $x$  e  $y$  identici, o fallimento  
  if  $\theta = \text{fallimento}$  then return fallimento  
  else if  $x = y$  then return  $\theta$   
  else if Variabile?( $x$ ) then return Unify-Var( $x, y, \theta$ )  
  else if Variabile?( $y$ ) then return Unify-Var( $y, x, \theta$ )  
  else if Composta?( $x$ ) and Composta?( $y$ ) then  
    return Unify(Args( $x$ ), Args( $y$ ), Unify(Op( $x$ ), Op( $y$ ),  $\theta$ ))  
  else if Lista?( $x$ ) and Lista?( $y$ ) then  
    return Unify(Resto( $x$ ), Resto( $y$ ), Unify(Primo( $x$ ), Primo( $y$ ),  $\theta$ ))  
  else return fallimento
```

una sostituzione,
inizialmente vuota

Algoritmo di unificazione (AIMA)

function Unify($x, y, \theta = \text{vuoto}$) **returns** una sostituzione che rende x e y identici, o *fallimento*

if $\theta = \text{fallimento}$ **then return** *fallimento*

else if $x = y$ **then return** θ

else if Variabile?(x) **then return** Unify-Var(x, y, θ)

else if Variabile?(y) **then return** Unify-Var(y, x, θ)

else if Composta?(x) **and** Composta?(y) **then**

return Unify(Args(x), Args(y), Unify(Op(x), Op(y), θ))

else if Lista?(x) **and** Lista?(y) **then**

return Unify(Resto(x), Resto(y), Unify(Primo(x), Primo(y), θ))

else return *fallimento*

Algoritmo di unificazione (AIMA)

```
function Unify( $x, y, \theta = \text{vuoto}$ ) returns una sostituzione che rende  $x$  e  $y$  identici, o fallimento  
    if  $\theta = \text{fallimento}$  then return fallimento  
    else if  $x = y$  then return  $\theta$   
    else if Variabile?( $x$ ) then return Unify-Var( $x, y, \theta$ )  
    else if Variabile?( $y$ ) then return Unify-Var( $y, x, \theta$ )  
    else if Composta?( $x$ ) and Composta?( $y$ ) then  
        return Unify(Args( $x$ ), Args( $y$ ), Unify(Op( $x$ ), Op( $y$ ),  $\theta$ ))  
    else if Lista?( $x$ ) and Lista?( $y$ ) then  
        return Unify(Resto( $x$ ), Resto( $y$ ), Unify(Primo( $x$ ), Primo( $y$ ),  $\theta$ ))  
    else return fallimento
```

se x o y sono variabili, chiama la
funzione Unify-Var

Algoritmo di unificazione (AIMA)

```
function Unify( $x, y, \theta = \text{vuoto}$ ) returns una sostituzione che rende  $x$  e  $y$  identici, o fallimento  
    if  $\theta = \text{fallimento}$  then return fallimento  
    else if  $x = y$  then return  $\theta$   
    else if Variabile?( $x$ ) then return Unify-Var( $x, y, \theta$ )  
    else if Variabile?( $y$ ) then return Unify-Var( $y, x, \theta$ )  
    else if Composta?( $x$ ) and Composta?( $y$ ) then  
        return Unify(Args( $x$ ), Args( $y$ ), Unify(Op( $x$ ), Op( $y$ ),  $\theta$ ))  
    else if Lista?( $x$ ) and Lista?( $y$ ) then  
        return Unify(Resto( $x$ ), Resto( $y$ ), Unify(Primo( $x$ ), Primo( $y$ ),  $\theta$ ))  
    else return fallimento
```

in questo caso x ed y sono
entrambe composte

es. se $x = F(A,B)$
Op restituisce F
Args restituisce (A,B)

Algoritmo di unificazione (AIMA)

```
function Unify( $x, y, \theta = \text{vuoto}$ ) returns una sostituzione che rende  $x$  e  $y$  identici, o fallimento  
    if  $\theta = \text{fallimento}$  then return fallimento  
    else if  $x = y$  then return  $\theta$   
    else if Variabile?( $x$ ) then return Unify-Var( $x, y, \theta$ )  
    else if Variabile?( $y$ ) then return Unify-Var( $y, x, \theta$ )  
    else if Composta?( $x$ ) and Composta?( $y$ ) then  
        return Unify(Args( $x$ ), Args( $y$ ), Unify(Op( $x$ ), Op( $y$ ),  $\theta$ ))  
    else if Lista?( $x$ ) and Lista?( $y$ ) then  
        return Unify(Resto( $x$ ), Resto( $y$ ), Unify(Primo( $x$ ), Primo( $y$ ),  $\theta$ ))  
    else return fallimento
```

Algoritmo di unificazione (cont.)

```
function Unify-Var(var, x,  $\theta$ ) returns una sostituzione  
  if  $\{var/val\} \in \theta$  per qualche val then return Unify(val, x,  $\theta$ )  
  else if  $\{x/val\} \in \theta$  per qualche val then return Unify(var, val,  $\theta$ )  
  else if Controlla-Occorrenza?(var, x) then return fallimento  
  else return Aggiungi  $\{var/x\}$  a  $\theta$ 
```

Controlla-Occorrenza controlla se *var* occorre all'interno dell'espressione *x* (occur check). In tal caso fallisce.

ATTENZIONE: *Aggiungi* non aggiunge semplicemente, ma applica la sostituzione in θ , altrimenti la sostituzione non sarebbe normalizzata.

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$

in questo caso x ed y sono
entrambe composte

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$

in questo caso x ed y sono
entrambe composte

```
else if Composta?(x) and Composta?(y) then  
    return Unify(Args(x), Args(y), Unify(Op(x), Op(y),  $\theta$ ))
```


Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$ **if $x = y$ then return θ**

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(A, x, \{ \}))$

if Lista?(x) **and** Lista?(y) **then**

return Unify(Resto(x), Resto(y), Unify(Primo(x), Primo(y), θ))

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(A, x, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY-VAR}(x, A, \{ \}))$

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(A, x, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY-VAR}(x, A, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \{x/A\})$

Aggiungi $\{var/x\}$ a θ

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
 - $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
 - $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$
 - $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(A, x, \{ \}))$
 - $\text{UNIFY}((y, z), (B, z), \text{UNIFY-VAR}(x, A, \{ \}))$
 - $\text{UNIFY}((y, z), (B, z), \{x/A\})$
 - $\text{UNIFY}((z), (z), \{y/B, x/A\})$
- simile procedimento per y e B

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(A, x, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY-VAR}(x, A, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \{x/A\})$
- $\text{UNIFY}((z), (z), \{y/B, x/A\})$
- $\text{UNIFY}(z, z, \{y/B, x/A\})$

if $x = y$ then return θ

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(A, x, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY-VAR}(x, A, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \{x/A\})$
- $\text{UNIFY}((z), (z), \{y/B, x/A\})$
- $\text{UNIFY}(z, z, \{y/B, x/A\})$
- $\{y/B, x/A\}$

Esempio 1: $P(A, y, z)$ e $P(x, B, z)$

- $\text{UNIFY}(P(A, y, z), P(x, B, z), \{ \})$
- $\text{UNIFY}((A, y, z), (x, B, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((A, y, z), (x, B, z), \{ \})$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(A, x, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY}(x, A, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \text{UNIFY-VAR}(x, A, \{ \}))$
- $\text{UNIFY}((y, z), (B, z), \{x/A\})$
- $\text{UNIFY}((z), (z), \{y/B, x/A\})$
- $\text{UNIFY}(z, z, \{y/B, x/A\})$
- $\{y/B, x/A\}$

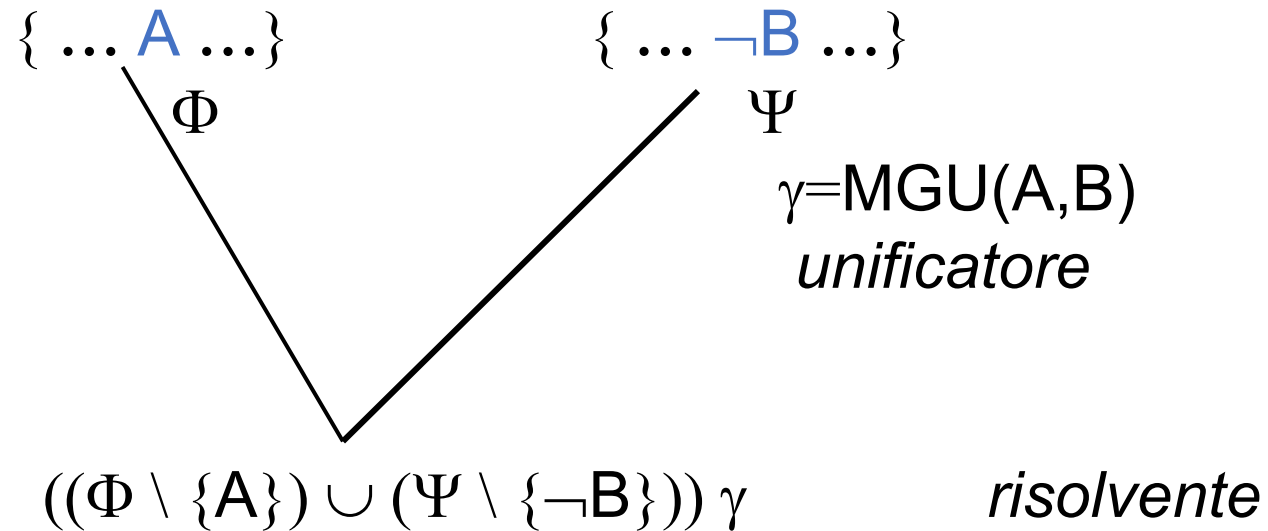
Esempio 2: $P(f(x), x)$ e $P(z, z)$

- $\text{UNIFY}(P(f(x), x), P(z, z), \{ \})$
- $\text{UNIFY}((f(x), x), (z, z), \text{UNIFY}(P, P, \{ \}))$
- $\text{UNIFY}((f(x), x), (z, z), \{ \})$
- $\text{UNIFY}((x), (z), \text{UNIFY}(f(x), z, \{ \}))$
- $\text{UNIFY}((x), (z), \text{UNIFY-VAR}(z, f(x), \{ \}))$
- $\text{UNIFY}((x), (z), \{z/f(x)\})$
- $\text{UNIFY-VAR}(x, z, \{z/f(x)\})$
- $\text{UNIFY}(x, f(x), \{z/f(x)\})$
- $\text{OCCUR-CHECK}(x, f(x))$
- *fallimento*

if $\{x/val\} \in \theta$ per qualche val then return $\text{Unify}(var, val, \theta)$

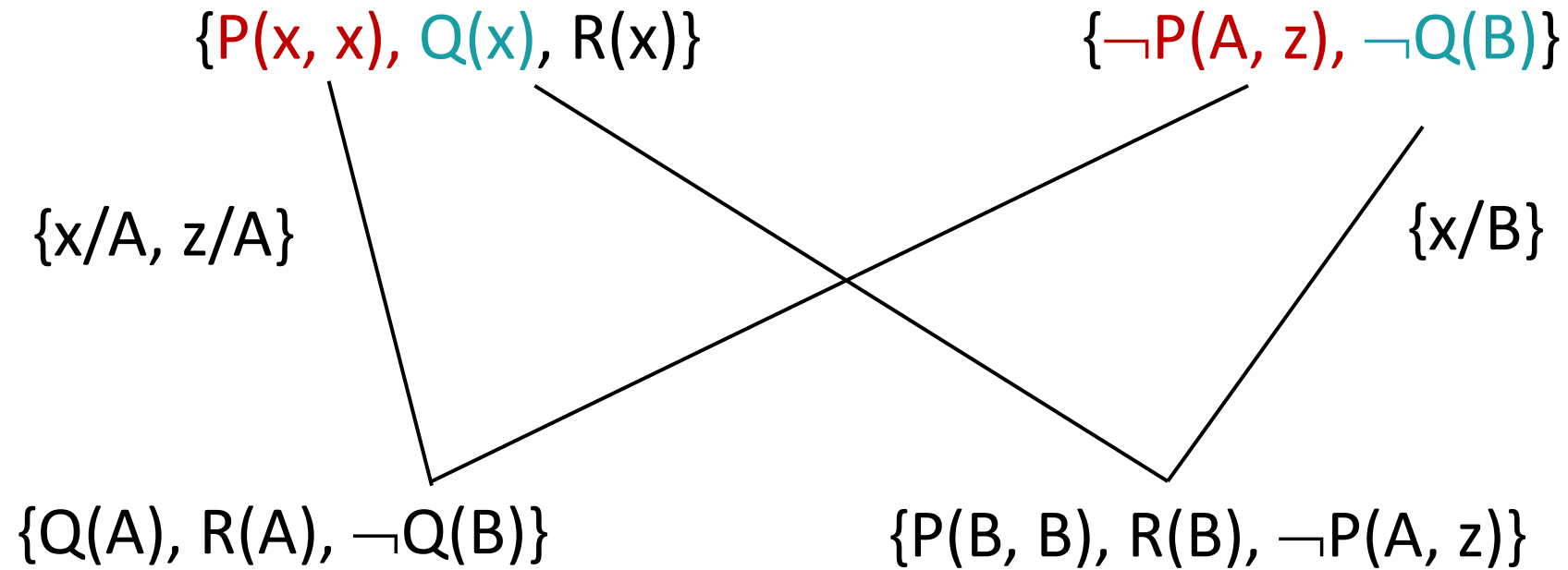
Siamo ora in grado di definire in generale la regola
di risoluzione per FOL

Il metodo di risoluzione per il FOL



N.B. $\setminus$ differenza di insiemi

Risoluzione: esempio



Grafo di risoluzione

Problema dei fattori

- Le seguenti clausole dovrebbero produrre la clausola vuota invece ...

$\{P(u), P(v)\}$

$\{\neg P(x), \neg P(y)\}$

$\{P(v), \neg P(y)\}$ e qui ci si ferma

- Se un sottoinsieme dei letterali di una stessa clausola può essere unificato allora la clausola ottenuta dopo tale unificazione si dice *fattore* della clausola originaria.
- Il metodo di risoluzione va applicato ai *fattori* delle clausole:

$\{P(u)\}$

$\{\neg P(x)\}$

$\{\}$

Completezza del metodo di risoluzione

- La deduzione per risoluzione **è corretta**

Correttezza: Se $KB \vdash_{\text{RES}} \alpha$ allora $KB \models \alpha$

- La deduzione per risoluzione **non è completa:**

può essere $KB \models \alpha$ e non $KB \vdash_{\text{RES}} \alpha$

Esempio

$\{ \} \models \{P, \neg P\}$ ma non $\{ \} \vdash_{\text{RES}} \{P, \neg P\}$

Risoluzione per *refutazione*

- Ancora, il *teorema di refutazione* ci suggerisce un metodo alternativo *completo*
- *Teorema di refutazione:*
 $KB \cup \{\neg\alpha\}$ è insoddisfacibile sse $KB \models A$
- *Teorema:* KB è insoddisfacibile sse $KB \vdash_{\text{RES}} \{ \}$
(la risoluzione è completa rispetto alla refutazione)
Abbiamo un metodo *meccanizzabile, corretto e completo*: basta aggiungere il negato della formula da dimostrare e provare a generare la clausola vuota

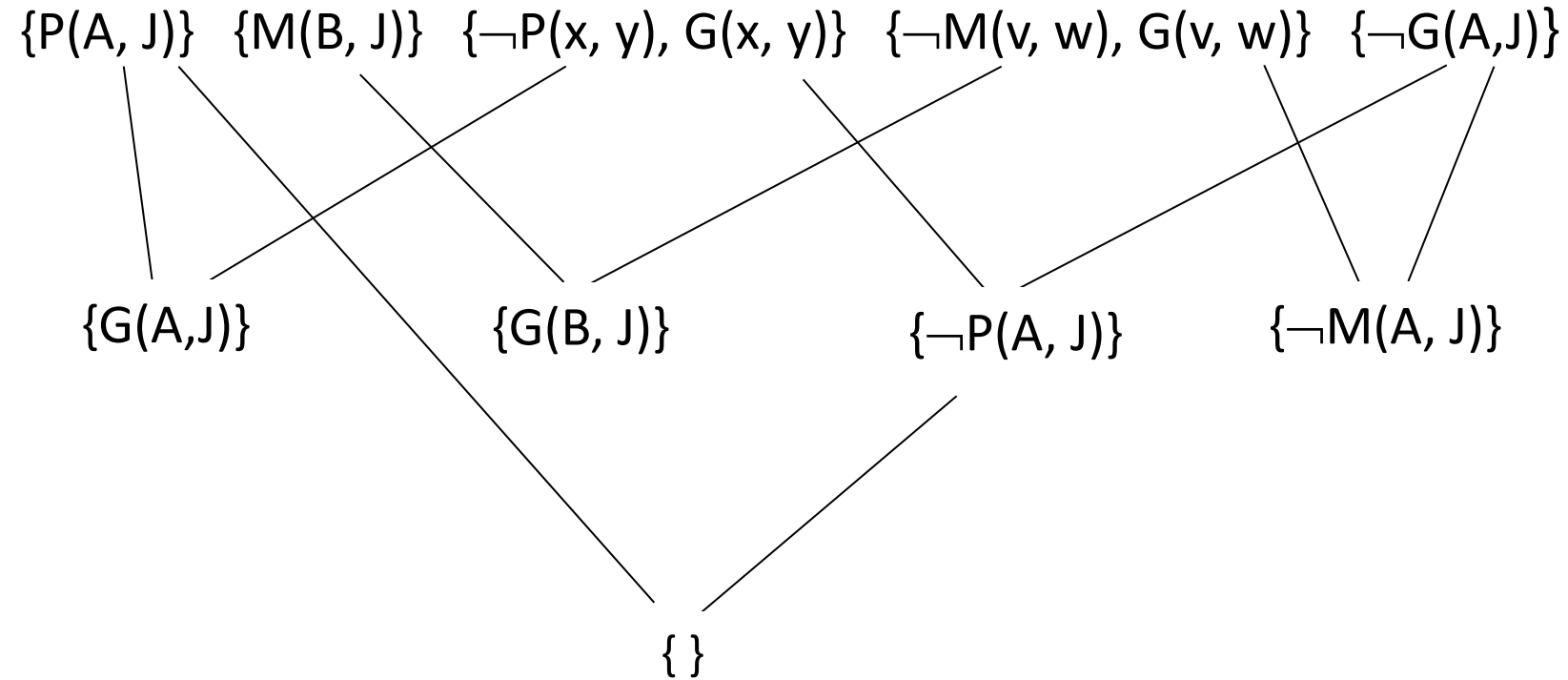
Esempio di refutazione

- | | | | | |
|----|-----------------------------|----|---------------------------|-------------------------------|
| 1. | $\{P(A, J)\}$ | |] <i>A è padre di J</i> | |
| 2. | $\{M(B, J)\}$ | | | |
| 3. | $\{\neg P(x, y), G(x, y)\}$ | KB | | <i>B è madre di J</i> |
| 4. | $\{\neg M(v, w), G(v, w)\}$ | | | <i>padre implica genitore</i> |
| ■ | <i>Goal: G(A, J)?</i> | | | <i>madre implica genitore</i> |
| | | | <i>A è genitore di J?</i> | |

Aggiungiamo a KB la negazione del goal e proviamo a dedurre la clausola vuota

5. $\{\neg G(A, J)\}$

Esempio di refutazione: il grafo



Refutazione per domande di tipo “trova ...”

- Esempio: “Chi sono i genitori di J?”
- Si cerca di dimostrare che $\exists z G(z, J)$

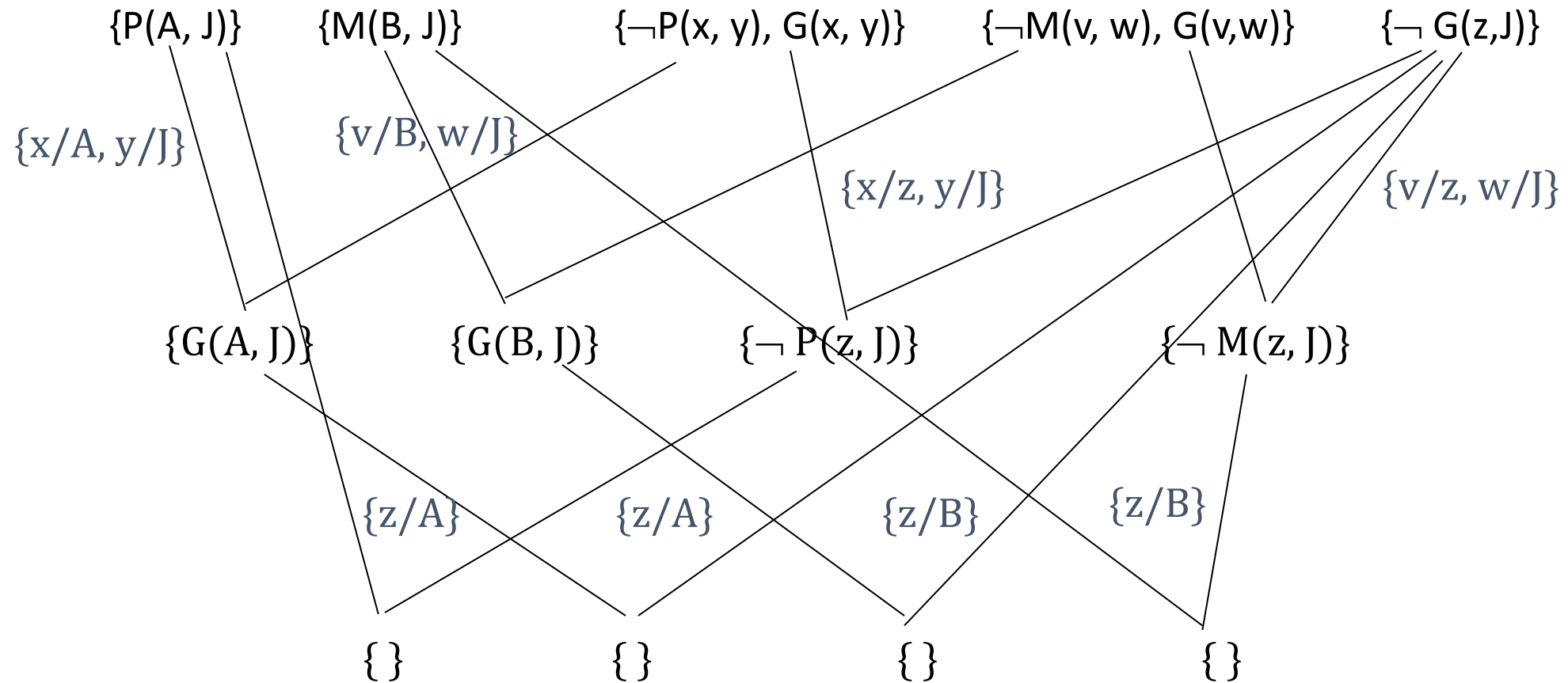
1. Si nega la clausola goal: $\neg \exists z G(z, J)$
2. Si porta in forma a clausole:

$$FC(\neg \exists z G(z, J)) \rightarrow FC(\forall z \neg G(z, J)) \rightarrow \{\neg G(z, J)\}$$

dove FC sta per “forma a clausole”

- La risposta sono tutti i possibili legami per z che consentono di ottenere la clausola vuota (**risposta calcolata**)

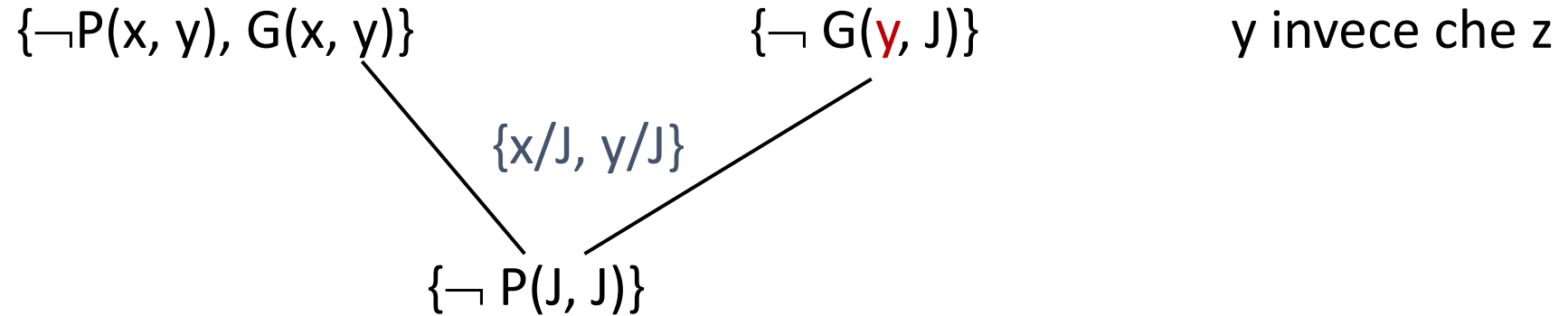
Esempio: chi sono i genitori di J?



Le risposte sono: A, B

Importante rinominare!

Osserva: è importante la restrizione che ogni clausola usi variabili diverse (anche quelle generate)



... e a questo punto non avremmo potuto ottenere la risposta unificando con $P(A, J)$